

1. Google Gemini Flash and Qwen 3.6 35B A3B
2. Gemini via <https://gemini.google.com/app>; Qwen via LM Studio
- 3.

Generate C++ code using objects, not functions, to implement the following requirements:

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap modules.
- Test File format:

Command Description

EMAIL <sender category>,
<subject line>,
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces,

but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

4.

Gemini:

```

1  #include <iostream>
2  #include <string>
3  #include <sstream>
4
5  // Represents an individual email
6  class Email {
7  private:
8      std::string sender;
9      std::string subject;
10     std::string date;
11
12     // Numerical rank for sender category (Higher = higher priority)
13     int getSenderRank() const {
14         if (sender == "Boss") return 5;
15         if (sender == "Subordinate") return 4;
16         if (sender == "Peer") return 3;
17         if (sender == "ImportantPerson") return 2;
18         if (sender == "OtherPerson") return 1;
19         return 0;
20     }
21
22     // Convert MM-DD-YYYY to an integer YYYYMMDD for straightforward chronological comparison
23     int getDateKey() const {
24         if (date.length() != 10) return 0;
25         int month = std::stoi(date.substr(0, 2));
26         int day = std::stoi(date.substr(3, 2));
27         int year = std::stoi(date.substr(6, 4));
28         return year * 10000 + month * 100 + day;
29     }
30
31 public:
32     Email() : sender(""), subject(""), date("") {}
33
34     Email(const std::string& sender, const std::string& subject, const std::string& date)
35         : sender(sender), subject(subject), date(date) {}
36
37     // Priority ordering:
38     // 1. Sender category priority
39     // 2. Tie-breaker: Newer date takes precedence (discovered at Sprint)
40     bool operator>(const Email& other) const {
41         int rankThis = getSenderRank();
42         int rankOther = other.getSenderRank();
43
44         if (rankThis != rankOther) {
45             return rankThis > rankOther;
46         }
47         return getDateKey() > other.getDateKey();
48     }
49
50     bool operator<(const Email& other) const {
51         return other > *this;
52     }
53
54     void display() const {
55         std::cout << "Sender: " << sender << "\n";
56         std::cout << "Subject: " << subject << "\n";
57         std::cout << "Date: " << date << "\n";
58     }
59 };
60
61 // MaxHeap implemented from scratch using a dynamically resizing array
62 class MaxHeap {
63 private:
64     Email* heap;
65     int capacity;
66     int currentSize;
67
68     void resize() {
69         int newCapacity = capacity * 2;
70         Email* newHeap = new Email[newCapacity];
71         for (int i = 0; i < currentSize; ++i) {
72             newHeap[i] = heap[i];
73         }
74         delete[] heap;
75         heap = newHeap;
76         capacity = newCapacity;
77     }

```

```

82         if (heap[index] > heap[parent]) {
83             swap(heap[index], heap[parent]);
84             index = parent;
85         } else {
86             break;
87         }
88     }
89 }
90
91 void siftDown(int index) {
92     while (true) {
93         int leftChild = 2 * index + 1;
94         int rightChild = 2 * index + 2;
95         int largest = index;
96
97         if (leftChild < currentSize && heap[leftChild] > heap[largest]) {
98             largest = leftChild;
99         }
100        if (rightChild < currentSize && heap[rightChild] > heap[largest]) {
101            largest = rightChild;
102        }
103
104        if (largest != index) {
105            swap(heap[index], heap[largest]);
106            index = largest;
107        } else {
108            break;
109        }
110    }
111 }
112
113 void swap(Email& a, Email& b) {
114     Email temp = a;
115     a = b;
116     b = temp;
117 }
118
119 public:
120     MaxHeap(int initialCapacity = 16)
121         : capacity(initialCapacity), currentSize(0) {
122         heap = new Email[capacity];
123     }
124
125     ~MaxHeap() {
126         delete[] heap;
127     }
128
129     void insert(const Email& email) {
130         if (currentSize == capacity) {
131             resize();
132         }
133         heap[currentSize] = email;
134         siftUp(currentSize);
135         currentSize++;
136     }
137
138     // Access the highest priority email without removing it
139     bool peek(Email& outEmail) const {
140         if (currentSize == 0) return false;
141         outEmail = heap[0];
142         return true;
143     }
144
145     // Delete the highest priority email
146     bool removeMax() {
147         if (currentSize == 0) return false;
148         heap[0] = heap[currentSize - 1];
149         currentSize--;
150         if (currentSize > 0) {
151             siftDown(0);
152         }

```

```

62  class MaxHeap {
119 public:
156     int size() const {
158     }
159
160     bool isEmpty() const {
161         return currentSize == 0;
162     }
163 };
164
165 // Priority Queue abstraction using the MaxHeap
166 class PriorityQueue {
167 private:
168     MaxHeap heap;
169
170 public:
171     void enqueue(const Email& email) {
172         heap.insert(email);
173     }
174
175     bool dequeue() {
176         return heap.removeMax();
177     }
178
179     bool peek(Email& email) const {
180         return heap.peek(email);
181     }
182
183     int count() const {
184         return heap.size();
185     }
186 };
187
188 // Command handler for processing CEO inbox instructions
189 class CEOInboxManager {
190 private:
191     PriorityQueue queue;
192
193     std::string trim(const std::string& str) {
194         size_t first = str.find_first_not_of(" \t\r\n");
195         if (first == std::string::npos) return "";
196         size_t last = str.find_last_not_of(" \t\r\n");
197         return str.substr(first, (last - first + 1));
198     }
199
200 public:
201     void processLine(const std::string& rawLine) {
202         std::string line = trim(rawLine);
203         if (line.empty()) return;
204
205         if (line.rfind("EMAIL", 0) == 0) {
206             // Extract the payload after the "EMAIL " keyword
207             std::string payload = line.substr(5);
208             std::stringstream ss(payload);
209             std::string sender, subject, date;
210
211             if (std::getline(ss, sender, ',') &&
212                 std::getline(ss, subject, ',') &&
213                 std::getline(ss, date)) {
214                 queue.enqueue(Email(trim(sender), trim(subject), trim(date)));
215             }
216         }
217         else if (line == "COUNT") {
218             std::cout << "There are " << queue.count() << " emails to read.\n";
219         }
220         else if (line == "NEXT") {
221             Email nextEmail;
222             if (queue.peek(nextEmail)) {
223                 std::cout << "Next email:\n";
224                 nextEmail.display();
225             } else {
226                 std::cout << "No emails to read.\n";
227             }
228         }
229         else if (line == "READ") {
230             queue.dequeue();
231         }

```

```
236         while (std::getline(in, line)) {
237             processLine(line);
238         }
239     }
240 };
241
242 int main() {
243     CEOInboxManager manager;
244     manager.run(std::cin);
245     return 0;
246 }
```

Qwen:

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <sstream>
5  #include <algorithm>
6
7  // Forward declaration of MaxHeap template
8  template <typename T>
9  class MaxHeap;
10
11  /**
12   * Email class to store email details and handle priority comparison.
13   */
14  class Email {
15  public:
16      std::string sender;
17      std::string subject;
18      std::string dateStr; // Original format MM-DD-YYYY
19
20      int year, month, day; // Parsed integers for easy comparison
21
22      Email() : year(0), month(0), day(0) {}
23
24      /**
25       * Parse the date string "MM-DD-YYYY" into integer components.
26       */
27      void parseDate(const std::string& d) {
28          dateStr = d;
29          // Format: MM-DD-YYYY
30          size_t dash1 = d.find('-');
31          size_t dash2 = d.find('-', dash1 + 1);
32
33          if (dash1 != std::string::npos && dash2 != std::string::npos) {
34              month = std::stoi(d.substr(0, dash1));
35              day = std::stoi(d.substr(dash1 + 1, dash2 - dash1 - 1));
36              year = std::stoi(d.substr(dash2 + 1));
37          } else {
38              // Fallback if format is wrong (though problem guarantees valid format)
39              month = 0; day = 0; year = 0;
40          }
41      }
42
43      /**
44       * Helper to get priority value for sender. Higher number = higher priority.
45       */
46      int getSenderPriority() const {
47          if (sender == "Boss") return 5;
48          if (sender == "Subordinate") return 4;
49          if (sender == "Peer") return 3;
50          if (sender == "ImportantPerson") return 2;
51          if (sender == "OtherPerson") return 1;
52          return 0; // Default for unknown senders, though problem limits to specific ones.
53      }
54
55      /**
56       * Comparison operator for MaxHeap.
57       * Returns true if 'this' is LESS than 'other'.
58       * Priority Logic:
59       * 1. Higher sender priority comes first (so we want lower value in < comparison).
60       * 2. If senders are equal, newer date comes first (so older date has lower value in < comparison).
61       */
62      bool operator<(const Email& other) const {
63          int myPriority = getSenderPriority();
64          int otherPriority = other.getSenderPriority();
65
66          if (myPriority != otherPriority) {
67              // Lower priority number means this email is "less" than the higher one.
68              return myPriority < otherPriority;
69          }
70
71          // If priorities are equal, compare dates.
72          // We want newer emails to be at the top of the MaxHeap (max element).
73          // So if 'this' is OLDER than 'other', 'this' should be "less" (<) than 'other'.
74
75          if (year != other.year) {
76              return year < other.year;

```

```

82     }
83
84     /**
85      * Output format for NEXT command.
86      */
87     void display() const {
88         std::cout << "Sender: " << sender << "\n";
89         std::cout << "Subject: " << subject << "\n";
90         std::cout << "Date: " << dateStr << "\n";
91     }
92 };
93
94 /**
95  * Generic MaxHeap implementation using a vector (list-based).
96  */
97 template <typename T>
98 class MaxHeap {
99 private:
100     std::vector<T> heap;
101
102     // Helper to swap elements
103     void swap(int i, int j) {
104         T temp = heap[i];
105         heap[i] = heap[j];
106         heap[j] = temp;
107     }
108
109     // Heapify up after insertion
110     void heapifyUp(int index) {
111         while (index > 0) {
112             int parent = (index - 1) / 2;
113             if (heap[index] < heap[parent]) { // Note: Using operator< defined in Email.
114                 // Wait, MaxHeap usually uses > for max heap?
115                 // Let's re-verify logic below.
116                 break;
117             }
118
119             // Standard MaxHeap Logic: Parent must be >= Child.
120             // If child (index) is GREATER than parent, swap them.
121             // So we check if heap[index] > heap[parent].
122             // But our Email operator< defines "less priority".
123             // We want the element with HIGHER priority to be at root.
124
125             // Let's stick to standard MaxHeap logic:
126             // If child is "greater" than parent, swap up.
127             // For Email, "greater" means higher sender priority OR newer date.
128             // Our operator< returns true if 'this' has LOWER priority/older date.
129             // So we need the inverse for swapping in a MaxHeap:
130             // If heap[index] is NOT less than heap[parent], it might be greater or equal.
131
132             // Actually, let's just implement standard comparison explicitly to avoid confusion with ope
133             if (isGreater(heap[index], heap[parent])) {
134                 swap(index, parent);
135                 index = parent;
136             } else {
137                 break;
138             }
139         }
140     }
141
142     // Heapify down after removal
143     void heapifyDown(int index) {
144         int size = heap.size();
145         while (true) {
146             int left = 2 * index + 1;
147             int right = 2 * index + 2;
148             int largest = index;
149
150             if (left < size && isGreater(heap[left], heap[largest])) {
151                 largest = left;
152             }
153             if (right < size && isGreater(heap[right], heap[largest])) {
154                 largest = right;
155             }
156
157             if (largest != index) {

```

```

160         } else {
161             break;
162         }
163     }
164 }
165
166 // Helper to determine if 'a' has higher priority than 'b' for Email type
167 bool isGreater(const T& a, const T& b) {
168     // If operator< is defined as "has lower priority", then ! (a < b) && !(b < a) implies equal.
169     // We want to know if a > b in terms of priority.
170     // Since Email::operator< returns true if 'this' is LOWER priority:
171     // a > b means !(a < b) AND NOT (a == b).
172     // However, simpler logic for MaxHeap with custom comparator:
173     // We want the "largest" element at top.
174     // Let's rely on the fact that Email::operator< is strictly "less priority".
175     // So 'greater' means it does not satisfy '<' and is not equal?
176     // Actually, let's just use a lambda or explicit check inside the class if needed,
177     // but since T is Email, we can cast.
178
179     // To keep this generic for any type that supports <, MaxHeap usually requires > logic.
180     // Let's assume standard behavior: MaxHeap puts max element at top.
181     // We will use a custom comparator approach or just rely on the fact that
182     // if T has operator< defined as "lower priority", we need to invert it for MaxHeap?
183
184     // Correction: The prompt asks for a MaxHeap.
185     // If Email::operator< means "is lower priority than", then the element with HIGHER priority is GREATER.
186     // So standard MaxHeap logic (swap if child > parent) works IF operator> is defined or we use !a<b.
187
188     // Let's implement explicit comparison for Email to be safe and clear.
189     const Email* eA = dynamic_cast<const Email*>(&a);
190     const Email* eB = dynamic_cast<const Email*>(&b);
191
192     if (eA && eB) {
193         return !(*eA < *eB) && !(*eB < *eA); // Strictly greater? No, that's equality.
194         // Let's use the explicit logic from Email::operator< again.
195         // If a has higher priority than b, then (b < a) is true.
196     }
197
198     // Generic fallback for other types if needed:
199     return b < a;
200 }
201
202 public:
203     void push(const T& item) {
204         heap.push_back(item);
205         heapifyUp(heap.size() - 1);
206     }
207
208     T pop() {
209         if (heap.empty()) {
210             throw std::runtime_error("Heap is empty");
211         }
212         T top = heap[0];
213         heap[0] = heap.back();
214         heap.pop_back();
215         if (!heap.empty()) {
216             heapifyDown(0);
217         }
218         return top;
219     }
220
221     bool empty() const {
222         return heap.empty();
223     }
224
225     size_t size() const {
226         return heap.size();
227     }
228
229     // Helper to peek at the max element without removing it (for NEXT command logic if needed,
230     // but we remove on READ. The prompt says NEXT displays then READ removes.
231     // Wait: "NEXT ... Display ... READ CEO has read email and dealt with it".
232     // Usually NEXT just shows top, READ removes top.
233     // But the sample output shows NEXT displaying, then READ removing.
234     // If I call NEXT twice, does it remove? No, usually NEXT is peek.
235     // Let's re-read: "NEXT ... Display the information ... READ CEO has read email and dealt with it".

```

```

236 // This implies NEXT is a Peek operation.
237
238 T peek() const {
239     if (heap.empty()) {
240         throw std::runtime_error("Heap is empty");
241     }
242     return heap[0];
243 }
244 };
245
246 /**
247  * CEOInbox class to manage the priority queue and process commands.
248  */
249 class CEOInbox {
250 private:
251     MaxHeap<Email> inbox;
252
253 public:
254     void addEmail(const std::string& sender, const std::string& subject, const std::string& date) {
255         Email e;
256         e.sender = sender;
257         e.subject = subject;
258         e.parseDate(date);
259         inbox.push(e);
260     }
261
262     void processNext() {
263         if (inbox.empty()) {
264             return; // Or handle error, but prompt implies valid commands or graceful handling.
265             // "be able to handle a file that does NEXT ... when there are no emails"
266         }
267
268         Email top = inbox.peek();
269         std::cout << "Next email:\n";
270         top.display();
271     }
272
273     void processRead() {
274         if (inbox.empty()) {
275             return; // "two READs in a row ... highest priority email deleted" -> implies if empty, do nothing.
276         }
277
278         inbox.pop();
279     }
280
281     void displayCount() {
282         std::cout << "There are " << inbox.size() << " emails to read.\n";
283     }
284
285     void processCommand(const std::string& command) {
286         if (command == "EMAIL") {
287             // This is handled by a separate parsing step in main because EMAIL has arguments.
288             // But the class structure suggests handling commands here.
289             // However, since EMAIL requires complex argument parsing from stdin,
290             // it's cleaner to parse in main and call addEmail.
291         } else if (command == "NEXT") {
292             processNext();
293         } else if (command == "READ") {
294             processRead();
295         } else if (command == "COUNT") {
296             displayCount();
297         }
298     }
299 };
300
301 /**
302  * Helper to parse the EMAIL command line.
303  */
304 void handleEmailCommand(std::istream& in, CEOInbox& inbox) {
305     std::string sender;
306     std::getline(in, sender, ','); // Read until comma
307
308     std::string subject;
309     std::getline(in, subject, ','); // Read until comma
310
311     std::string date;
312     std::getline(in, date); // Read rest of line (date)
313

```

```

314     // Trim potential whitespace/newlines from date if necessary
315     if (!date.empty() && date.back() == '\r') {
316         date.pop_back();
317     }
318
319     inbox.addEmail(sender, subject, date);
320 }
321
322 int main() {
323     CEOInbox inbox;
324     std::string command;
325
326     while (std::cin >> command) {
327         if (command == "EMAIL") {
328             handleEmailCommand(std::cin, inbox);
329         } else if (command == "NEXT") {
330             inbox.processNext();
331         } else if (command == "READ") {
332             inbox.processRead();
333         } else if (command == "COUNT") {
334             inbox.displayCount();
335         }
336     }
337
338     return 0;
339 }

```

5.

a. Gemini

- i. Correctness: Handles the happy path correctly. When READ is used before emails are added, no message is output. When NEXT is used before emails are added however, a message does appear.
- ii. Execution Speed: Everytime an email needs to be checked for its priority, 5 string comparisons may be executed, versus just storing the category as an integer in the email object. Also converts date to an integer on every comparison, versus pre-computing on email creation. In the swap function, emails are swapped by copies, versus using the swap function to change pointers.
- iii. Space Complexity: Email data is stored as dynamically allocated strings, versus integers and one string.
- iv. Maintainability: Category parsing is hard-coded, versus using constants. Comments are lacking in some areas, but present to

some degree. Bad inputs are silently dropped, instead of outputting an error message. Does not use C++ standard objects, which can offer more safety (such as vectors and unique pointers), although this may not follow assignment guidelines.

b. Qwen

- i. Correctness: Does not compile. Uses `size_t`, which is not included by default. If `cstdint` is included to fix compilation error, code outputs wrong order. The `isGreater` function returns a boolean expression, which simplifies to an equality comparison, returning false if the emails are not equal. When an email is read in, whitespace isn't consumed when appropriate, which puts leading whitespace in the sender field, which causes `getSenderPriority` to always return 0.
- ii. Execution Speed: Pre-parses the date into integers when an email is added to reduce recomputations. In the swap function, emails are swapped with copies, versus using the swap function. When an email's sender priority is checked, 5 string comparisons may be executed, instead of using the same logic with date parsing, doing so ahead of time.
- iii. Space Complexity: `peek` function returns by value, instead of by reference. Memory that was occupied by popped emails is never returned, leading to a memory leak.
- iv. Maintainability: Comments are plentiful, although some are streams of consciousness, rather than being helpful. `MaxHeap` is designed to be generic, but also includes logic specific to email objects. `processCommand` for `CEOInbox` does not handle EMAIL commands itself, even though it handles everything else. Category parsing is hard-coded, versus using constants. Bad inputs are silently dropped, instead of outputting an error message.

6. I choose to use Gemini for this assignment, because the code can correctly follow the happy path, and the issues found are not difficult to fix.

7.

- a. Correctness: Added an error message when READ is executed while there's no emails available to read.
- b. Execution Speed: Converted sender to an enum that is determined when an email is created. Changed swap function to use move semantics.
- c. Space Complexity: Changed dates to be stored as 3 integers.
- d. Maintainability: Added error message for unrecognized command.